

Quantum Computing and Quantum Machine Learning

Morten Hjorth-Jensen¹

Department of Physics, University of Oslo, Norway¹

April 23

© 1999-2025, Morten Hjorth-Jensen. Released under CC Attribution-NonCommercial 4.0 license

Plans for the week of April 21-25

Quantum Machine Learning (QML)

1. Introduction to QML
2. Support Vector Machines (SVM, classical machine learning approach)
3. Quantum Support Vector Machine Learning (QSVM)

Reading recommendations

1. For classical Support Vector Machines, see lecture notes by Morten Hjorth-Jensen at https://compphysics.github.io/MachineLearning/doc/LectureNotes/_build/html/chapter5.html
2. Claudio Conti, Quantum Machine Learning (Springer), sections 1.5-1.12, see <https://link.springer.com/book/10.1007/978-3-031-44226-1>. With an IP-number from the university of Oslo, you can download this book for free. The material on quantum support vector machines will be discussed next week.

What is Machine Learning?

Machine Learning (ML) is the study of algorithms that improve through data experience.

Types of Machine Learning:

1. **Supervised Learning:** Labeled data for classification or regression.
 2. **Unsupervised Learning:** No labels; discover hidden patterns.
- o**Reinforcement Learning:** Learning through interaction with the environment.

ML Workflow:

Data → Model Training → Prediction

What is Quantum Machine Learning?

Quantum Machine Learning (QML) integrates quantum computing with machine learning algorithms to exploit quantum advantages.

Motivation:

1. High-dimensional Hilbert spaces for better feature representation.
2. Quantum parallelism for faster computation.
3. Quantum entanglement for richer data encoding.

Quantum Speedups in ML

Why Quantum?

1. **Quantum Parallelism:** Process multiple states simultaneously.
2. **Quantum Entanglement:** Correlated states for richer information.
3. **Quantum Interference:** Constructive and destructive interference to enhance solutions.

Example - Quantum Fourier transforms:

$$\text{DiscreteFourierTransform} \sim O(N^2)$$

$$\text{FastFourierTransform} \sim O(N^2)$$

$$\text{QuantumFourierTransform} \sim O(N)$$

Challenges in Quantum Machine Learning

Quantum Hardware Limitations:

1. Noisy Intermediate-Scale Quantum (NISQ) devices.
2. Decoherence and limited qubit coherence times.

Data Encoding:

1. Efficient embedding of classical data into quantum states.

Scalability:

1. Difficult to scale circuits to large datasets.

Classical Support Vector Machines (SVMs)

Support Vector Machines (SVM) are supervised learning algorithms used for classification tasks, and regression tasks as well. The main goal of SVMs is to find the optimal separating hyperplane (in high-dimensional space) that provides a maximum margin between classes.

First mathematical formulation of SVMs (formal details below)

For a dataset (x_i, y_i) where $x_i \in \mathbb{R}^n$ and $y_i \in \{-1, 1\}$, the decision boundary is defined as:

$$f(x) = w \cdot x + b = 0.$$

The goal is to optimize

$$\min_{w,b} \frac{1}{2} \|w\|^2,$$

subject to

$$y_i(w \cdot x_i + b) \geq 1, \quad \forall i$$

Kernels and more

In classical SVMs, kernels help with non-linear data separations. Quantum computers can speed up the computation of complex kernel evaluations by efficiently simulating an inner product in an exponentially large Hilbert space.

Quantum Support Vector Machines (QSVMs)

Quantum Kernel Estimation:

1. Maps classical data to a quantum Hilbert space.
2. Quantum kernel measures similarity in high-dimensional space.

Quantum-enhanced kernel:

$$K(\mathbf{x}, \mathbf{x}') = |\langle \psi(\mathbf{x}) | \psi(\mathbf{x}') \rangle|^2$$

Here, $|\phi(\mathbf{x})\rangle$ is the quantum state encoding of the classical data $\mathbf{x}$.
Advantage: Potentially exponential speedup over classical SVMs.

Quantum Neural Networks (QNNs)

Quantum Neural Networks replace classical neurons with parameterized quantum circuits.

Key Concepts:

1. Quantum Gates as Activation Functions.
2. Variational Quantum Circuits (VQCs) for optimization.

Parameterized Quantum Circuit:

$$U(\theta) = \prod_i R_y(\theta_i) \cdot CNOT \cdot R_x(\theta_i)$$

Advantage: Quantum gradients enable exploration of non-convex landscapes.

Quantum Boltzmann Machines (QBMs)

Quantum Boltzmann Machines leverage quantum mechanics to sample from a probability distribution.

1. Quantum tunneling aids in escaping local minima.
2. Quantum annealing for optimization problems.

Quantum Hamiltonian:

$$H = - \sum_i b_i \sigma_i^z - \sum_{ij} w_{ij} \sigma_i^z \sigma_j^z$$

Advantage: Efficient sampling in complex probability distributions.

Classical Support Vector Machines, overarching aims (best visualized with jupyter-notebook)

A Support Vector Machine (SVM) is a very powerful and versatile Machine Learning method, capable of performing linear or nonlinear classification, regression, and even outlier detection. It is one of the most popular models in Machine Learning, and anyone interested in Machine Learning should have it in their toolbox. SVMs are particularly well suited for classification of complex but small-sized or medium-sized datasets.

The case with two well-separated classes only can be understood in an intuitive way in terms of lines in a two-dimensional space separating the two classes (see figure below).

Basic mathematics

The basic mathematics behind the SVM is however less familiar to most of us. It relies on the definition of hyperplanes and the definition of a **margin** which separates classes (in case of classification problems) of variables. It is also used for regression problems.

With SVMs we distinguish between hard margin and soft margins. The latter introduces a so-called softening parameter to be discussed below. We distinguish also between linear and non-linear approaches. The latter are the most frequent ones since it is rather unlikely that we can separate classes easily by say straight lines.

Hyperplanes and all that

The theory behind support vector machines (SVM hereafter) is based on the mathematical description of so-called hyperplanes. Let us start with a two-dimensional case. This will also allow us to introduce our first SVM examples. These will be tailored to the case of two specific classes, as displayed in the figure here based on the usage of the petal data.

We assume here that our data set can be well separated into two domains, where a straight line does the job in the separating the two classes. Here the two classes are represented by either squares or circles.

```
from sklearn import datasets
from sklearn.svm import SVC, LinearSVC
from sklearn.linear_model import SGDClassifier
from sklearn.preprocessing import StandardScaler
import matplotlib
import matplotlib.pyplot as plt
plt.rcParams['axes.labelsize'] = 14
plt.rcParams['xtick.labelsize'] = 12
plt.rcParams['ytick.labelsize'] = 12

iris = datasets.load_iris()
X = iris["data"][:, (2, 3)]  # petal length, petal width
```

What is a hyperplane?

The aim of the SVM algorithm is to find a hyperplane in a p -dimensional space, where p is the number of features that distinctly classifies the data points.

In a p -dimensional space, a hyperplane is what we call an affine subspace of dimension of $p - 1$. As an example, in two dimension, a hyperplane is simply as straight line while in three dimensions it is a two-dimensional subspace, or stated simply, a plane.

Two-dimensional case

In two dimensions, with the variables x_1 and x_2 , the hyperplane is defined as

$$b + w_1x_1 + w_2x_2 = 0,$$

where b is the intercept and w_1 and w_2 define the elements of a vector orthogonal to the line $b + w_1x_1 + w_2x_2 = 0$. In two dimensions we define the vectors $\mathbf{x} = [x_1, x_2]$ and $\mathbf{w} = [w_1, w_2]$. We can then rewrite the above equation as

$$\mathbf{x}^T \mathbf{w} + b = 0.$$

A p -dimensional space of features

We limit ourselves to two classes of outputs y_i and assign these classes the values $y_i = \pm 1$. In a p -dimensional space of say p features we have a hyperplane defines as

$$b + wx_1 + w_2x_2 + \cdots + w_px_p = 0.$$

If we define a matrix $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_p]$ of dimension $n \times p$, where n represents the observations for each feature and each vector $\mathbf{x}_i$ is a column vector of the matrix $\mathbf{X}$,

$$\mathbf{x}_i = \begin{bmatrix} x_{i1} \\ x_{i2} \\ \dots \\ \dots \\ x_{ip} \end{bmatrix}.$$

More details

If the above condition is not met for a given vector $\mathbf{x}_i$ we have

$$b + w_1x_{i1} + w_2x_{i2} + \cdots + w_px_{ip} > 0,$$

if our output $y_i = 1$. In this case we say that $\mathbf{x}_i$ lies on one of the sides of the hyperplane and if

$$b + w_1x_{i1} + w_2x_{i2} + \cdots + w_px_{ip} < 0,$$

for the class of observations $y_i = -1$, then $\mathbf{x}_i$ lies on the other side. Equivalently, for the two classes of observations we have

$$y_i (b + w_1x_{i1} + w_2x_{i2} + \cdots + w_px_{ip}) > 0.$$

When we try to separate hyperplanes, if it exists, we can use it to construct a natural classifier: a test observation is assigned a given class depending on which side of the hyperplane it is located.

The two-dimensional case

Let us try to develop our intuition about SVMs by limiting ourselves to a two-dimensional plane. To separate the two classes of data points, there are many possible lines (hyperplanes if you prefer a more strict naming) that could be chosen. Our objective is to find a plane that has the maximum margin, i.e the maximum distance between data points of both classes. Maximizing the margin distance provides some reinforcement so that future data points can be classified with more confidence.

Linear classifier

What a linear classifier attempts to accomplish is to split the feature space into two half spaces by placing a hyperplane between the data points. This hyperplane will be our decision boundary. All points on one side of the plane will belong to class one and all points on the other side of the plane will belong to the second class two.

Unfortunately there are many ways in which we can place a hyperplane to divide the data. Below is an example of two candidate hyperplanes for our data sample.

Getting into the details

Let us define the function

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b = 0,$$

as the function that determines the line L that separates two classes (our two features), see the figure here.

Any point defined by $\mathbf{x}_1$ and $\mathbf{x}_2$ on the line L will satisfy $\mathbf{w}^T(\mathbf{x}_1 - \mathbf{x}_2) = 0$.

The signed distance δ from any point defined by a vector $\mathbf{x}$ and a point $\mathbf{x}_0$ on the line L is then

$$\delta = \frac{1}{\|\mathbf{w}\|}(\mathbf{w}^T \mathbf{x} + b).$$

First attempt at a minimization approach

How do we find the parameter b and the vector $\mathbf{w}$? What we could do is to define a cost function which now contains the set of all misclassified points M and attempt to minimize this function

$$C(\mathbf{w}, b) = - \sum_{i \in M} y_i (\mathbf{w}^T \mathbf{x}_i + b).$$

We could now for example define all values $y_i = 1$ as misclassified in case we have $\mathbf{w}^T \mathbf{x}_i + b < 0$ and the opposite if we have $y_i = -1$. Taking the derivatives gives us

$$\frac{\partial C}{\partial b} = - \sum_{i \in M} y_i,$$

and

$$\frac{\partial C}{\partial \mathbf{w}} = - \sum_{i \in M} y_i \mathbf{x}_i.$$

Solving the equations

We can now use the Newton-Raphson method or different variants of the gradient descent family (from plain gradient descent to various stochastic gradient descent approaches) to solve the equations

$$b \leftarrow b + \eta \frac{\partial C}{\partial b},$$

and

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \frac{\partial C}{\partial \mathbf{w}},$$

where η is our by now well-known learning rate.

Problems with the Simpler Approach

The equations we discussed above can be coded rather easily (the framework is similar to what has been developed for say logistic regression).

There are however problems with this approach, although it looks pretty straightforward to implement. When running such a code, we see that we can easily end up with many different lines which separate the two classes.

For small gaps between the entries, we may also end up needing many iterations before the solutions converge and if the data cannot be separated properly into two distinct classes, we may not experience a converge at all.

A better approach

A better approach is rather to try to define a large margin between the two classes (if they are well separated from the beginning).

Thus, we wish to find a margin M with $\mathbf{w}$ normalized to $\|\mathbf{w}\| = 1$ subject to the condition

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq M \quad \forall i = 1, 2, \dots, p.$$

All points are thus at a signed distance from the decision boundary defined by the line L . The parameters b and w_1 and w_2 define this line.

Largest value M

We seek the largest value M defined by

$$\frac{1}{\|\mathbf{w}\|} y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq M \quad \forall i = 1, 2, \dots, n,$$

or just

$$y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq M \|\mathbf{w}\| \quad \forall i.$$

If we scale the equation so that $\|\mathbf{w}\| = 1/M$, we have to find the minimum of $\mathbf{w}^T \mathbf{w} = \|\mathbf{w}\|^2$ (the norm) subject to the condition

$$y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i.$$

We have thus defined our margin as the invers of the norm of $\mathbf{w}$. We want to minimize the norm in order to have a as large as possible margin M . Before we proceed, we need to remind ourselves about Lagrangian multipliers.

A quick Reminder on Lagrangian Multipliers

Consider a function of three independent variables $f(x, y, z)$. For the function f to be an extreme we have

$$df = 0.$$

A necessary and sufficient condition is

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial y} = \frac{\partial f}{\partial z} = 0,$$

due to

$$df = \frac{\partial f}{\partial x} dx + \frac{\partial f}{\partial y} dy + \frac{\partial f}{\partial z} dz.$$

Not all variables are independent of each other

In many problems the variables x, y, z are often subject to constraints (such as those above for the margin) so that they are no longer all independent. It is possible at least in principle to use each constraint to eliminate one variable and to proceed with a new and smaller set of independent variables.

The use of so-called Lagrangian multipliers is an alternative technique when the elimination of variables is inconvenient or undesirable. Assume that we have an equation of constraint on the variables x, y, z

$$\phi(x, y, z) = 0,$$

resulting in

$$d\phi = \frac{\partial\phi}{\partial x}dx + \frac{\partial\phi}{\partial y}dy + \frac{\partial\phi}{\partial z}dz = 0.$$

Only two independent variables

Now we cannot set anymore

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial y} = \frac{\partial f}{\partial z} = 0,$$

if $df = 0$ is wanted because there are now only two independent variables! Assume x and y are the independent variables. Then dz is no longer arbitrary.

Adding the Multiplier

However, we can add to

$$df = \frac{\partial f}{\partial x} dx + \frac{\partial f}{\partial y} dy + \frac{\partial f}{\partial z} dz,$$

a multiple of $d\phi$, viz. $\lambda d\phi$, resulting in

$$df + \lambda d\phi = \left(\frac{\partial f}{\partial x} + \lambda \frac{\partial \phi}{\partial x}\right) dx + \left(\frac{\partial f}{\partial y} + \lambda \frac{\partial \phi}{\partial y}\right) dy + \left(\frac{\partial f}{\partial z} + \lambda \frac{\partial \phi}{\partial z}\right) dz = 0.$$

Our multiplier is chosen so that

$$\frac{\partial f}{\partial z} + \lambda \frac{\partial \phi}{\partial z} = 0.$$

More details

We need to remember that we took dx and dy to be arbitrary and thus we must have

$$\frac{\partial f}{\partial x} + \lambda \frac{\partial \phi}{\partial x} = 0,$$

and

$$\frac{\partial f}{\partial y} + \lambda \frac{\partial \phi}{\partial y} = 0.$$

When all these equations are satisfied, $df = 0$. We have four unknowns, x, y, z and λ . Actually we want only x, y, z , λ needs not to be determined, it is therefore often called Lagrange's undetermined multiplier. If we have a set of constraints ϕ_k we have the equations

$$\frac{\partial f}{\partial x_i} + \sum_k \lambda_k \frac{\partial \phi_k}{\partial x_i} = 0.$$

Setting up the problem

In order to solve the above problem, we define the following Lagrangian function to be minimized

$$\mathcal{L}(\lambda, b, \mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} - \sum_{i=1}^n \lambda_i \left[y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 \right],$$

where λ_i is a so-called Lagrange multiplier subject to the condition $\lambda_i \geq 0$.

Setting up derivatives

Taking the derivatives with respect to b and $\mathbf{w}$ we obtain

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_i \lambda_i y_i = 0,$$

and

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 = \mathbf{w} - \sum_i \lambda_i y_i \mathbf{x}_i.$$

Inserting these constraints into the equation for $\mathcal{L}$ we obtain

$$\mathcal{L} = \sum_i \lambda_i - \frac{1}{2} \sum_{ij}^n \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j,$$

subject to the constraints $\lambda_i \geq 0$ and $\sum_i \lambda_i y_i = 0$.

Karush-Kuhn-Tucker condition

We must in addition satisfy the **Karush-Kuhn-Tucker** (KKT) condition

$$\lambda_i \left[y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \right] \forall i.$$

1. If $\lambda_i > 0$, then $y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$ and we say that $\mathbf{x}_i$ is on the boundary.
2. If $y_i(\mathbf{w}^T \mathbf{x}_i + b) > 1$, we say $\mathbf{x}_i$ is not on the boundary and we set $\lambda_i = 0$.

When $\lambda_i > 0$, the vectors $\mathbf{x}_i$ are called support vectors. They are the vectors closest to the line (or hyperplane) and define the margin M .

The problem to solve

We can rewrite

$$\mathcal{L} = \sum_i \lambda_i - \frac{1}{2} \sum_{ij}^n \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j,$$

and its constraints in terms of a matrix-vector problem where we minimize w.r.t. λ the following problem

$$\frac{1}{2} \boldsymbol{\lambda}^T \begin{bmatrix} y_1 y_1 \mathbf{x}_1^T \mathbf{x}_1 & y_1 y_2 \mathbf{x}_1^T \mathbf{x}_2 & \dots & \dots & y_1 y_n \mathbf{x}_1^T \mathbf{x}_n \\ y_2 y_1 \mathbf{x}_2^T \mathbf{x}_1 & y_2 y_2 \mathbf{x}_2^T \mathbf{x}_2 & \dots & \dots & y_2 y_n \mathbf{x}_2^T \mathbf{x}_n \\ \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots \\ y_n y_1 \mathbf{x}_n^T \mathbf{x}_1 & y_n y_2 \mathbf{x}_n^T \mathbf{x}_2 & \dots & \dots & y_n y_n \mathbf{x}_n^T \mathbf{x}_n \end{bmatrix} \boldsymbol{\lambda} - \mathbb{1}^T \boldsymbol{\lambda},$$

subject to $\mathbf{y}^T \boldsymbol{\lambda} = 0$. Here we defined the vectors

$\boldsymbol{\lambda} = [\lambda_1, \lambda_2, \dots, \lambda_n]$ and $\mathbf{y} = [y_1, y_2, \dots, y_n]$.

The last steps

Solving the above problem, yields the values of λ_i . To find the coefficients of your hyperplane we need simply to compute

$$\mathbf{w} = \sum_i \lambda_i y_i \mathbf{x}_i.$$

With our vector $\mathbf{w}$ we can in turn find the value of the intercept b (here in two dimensions) via

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1,$$

resulting in

$$b = \frac{1}{y_i} - \mathbf{w}^T \mathbf{x}_i,$$

or if we write it out in terms of the support vectors only, with N_s being their number, we have

$$b = \frac{1}{N_s} \sum_{j \in N_s} \left(y_j - \sum_{i=1}^n \lambda_i y_i \mathbf{x}_i^T \mathbf{x}_j \right).$$

Classifier equations

With our hyperplane coefficients we can use our classifier to assign any observation by simply using

$$y_i = \text{sign}(\mathbf{w}^T \mathbf{x}_i + b).$$

Below we discuss how to find the optimal values of λ_j . Before we proceed however, we discuss now the so-called soft classifier.

A soft classifier

Till now, the margin is strictly defined by the support vectors. This defines what is called a hard classifier, that is the margins are well defined.

Suppose now that classes overlap in feature space, as shown in the figure here. One way to deal with this problem before we define the so-called **kernel approach**, is to allow a kind of slack in the sense that we allow some points to be on the wrong side of the margin. We introduce thus the so-called **slack** variables $\xi = [\xi_1, \xi_2, \dots, \xi_n]$ and modify our previous equation

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1,$$

to

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1 - \xi_i,$$

with the requirement $\xi_i \geq 0$. The total violation is now $\sum_i \xi_i$.

Misclassification

The value ξ_i in the constraint the last constraint corresponds to the amount by which the prediction $y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$ is on the wrong side of its margin. Hence by bounding the sum $\sum_i \xi_i$, we bound the total amount by which predictions fall on the wrong side of their margins.

Misclassifications occur when $\xi_i > 1$. Thus bounding the total sum by some value C bounds in turn the total number of misclassifications.

Soft optimization problem

This has in turn the consequences that we change our optimization problem to finding the minimum of

$$\mathcal{L} = \frac{1}{2} \mathbf{w}^T \mathbf{w} - \sum_{i=1}^n \lambda_i \left[y_i (\mathbf{w}^T \mathbf{x}_i + b) - (1 - \xi_i) \right] + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \gamma_i \xi_i,$$

subject to

$$y_i (\mathbf{w}^T \mathbf{x}_i + b) = 1 - \xi_i \quad \forall i,$$

with the requirement $\xi_i \geq 0$.

Derivatives with respect to b and $\mathbf{w}$

Taking the derivatives with respect to b and $\mathbf{w}$ we obtain

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_i \lambda_i y_i = 0,$$

and

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 = \mathbf{w} - \sum_i \lambda_i y_i \mathbf{x}_i,$$

and

$$\lambda_i = C - \gamma_i \forall i.$$

New constraints

Inserting these constraints into the equation for $\mathcal{L}$ we obtain the same equation as before

$$\mathcal{L} = \sum_i \lambda_i - \frac{1}{2} \sum_{ij}^n \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j,$$

but now subject to the constraints $\lambda_i \geq 0$, $\sum_i \lambda_i y_i = 0$ and $0 \leq \lambda_i \leq C$. We must in addition satisfy the Karush-Kuhn-Tucker condition which now reads

$$\lambda_i \left[y_i (\mathbf{w}^T \mathbf{x}_i + b) - (1 - \xi_i) \right] = 0 \quad \forall i,$$

$$\gamma_i \xi_i = 0,$$

and

$$y_i (\mathbf{w}^T \mathbf{x}_i + b) - (1 - \xi_i) \geq 0 \quad \forall i.$$

Kernels and non-linearity

The cases we have studied till now, were all characterized by two classes with a close to linear separability. The classifiers we have described so far find linear boundaries in our input feature space. It is possible to make our procedure more flexible by exploring the feature space using other basis expansions such as higher-order polynomials, wavelets, splines etc.

If our feature space is not easy to separate, as shown in the figure here, we can achieve a better separation by introducing more complex basis functions. The ideal would be, as shown in the next figure, to, via a specific transformation to obtain a separation between the classes which is almost linear.

The change of basis, from $x \rightarrow z = \phi(x)$ leads to the same type of equations to be solved, except that we need to introduce for example a polynomial transformation to a two-dimensional training set.

```
import numpy as np
import os
```

```
np.random.seed(42)
```

```
# To plot pretty figures
```

The equations

Suppose we define a polynomial transformation of degree two only (we continue to live in a plane with x_i and y_i as variables)

$$\mathbf{z} = \phi(\mathbf{x}_i) = \left(x_i^2, y_i^2, \sqrt{2}x_i y_i \right).$$

With our new basis, the equations we solved earlier are basically the same, that is we have now (without the slack option for simplicity)

$$\mathcal{L} = \sum_i \lambda_i - \frac{1}{2} \sum_{ij}^n \lambda_i \lambda_j y_i y_j \mathbf{z}_i^T \mathbf{z}_j,$$

subject to the constraints $\lambda_i \geq 0$, $\sum_i \lambda_i y_i = 0$, and for the support vectors

$$y_i(\mathbf{w}^T \mathbf{z}_i + b) = 1 \quad \forall i,$$

from which we also find b .

Defining the kernel

To compute $\mathbf{z}_i^T \mathbf{z}_j$ we define the kernel $K(\mathbf{x}_i, \mathbf{x}_j)$ as

$$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{z}_i^T \mathbf{z}_j = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j).$$

For the above example, the kernel reads

$$K(\mathbf{x}_i, \mathbf{x}_j) = [x_i^2, y_i^2, \sqrt{2}x_i y_i]^T \begin{bmatrix} x_j^2 \\ y_j^2 \\ \sqrt{2}x_j y_j \end{bmatrix} = x_i^2 x_j^2 + 2x_i x_j y_i y_j + y_i^2 y_j^2.$$

Kernel trick

We note that this is nothing but the dot product of the two original vectors $(\mathbf{x}_i^T \mathbf{x}_j)^2$. Instead of thus computing the product in the Lagrangian of $\mathbf{z}_i^T \mathbf{z}_j$ we simply compute the dot product $(\mathbf{x}_i^T \mathbf{x}_j)^2$. This leads to the so-called kernel trick and the result leads to the same as if we went through the trouble of performing the transformation $\phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$ during the SVM calculations.

The problem to solve

Using our definition of the kernel We can rewrite again the Lagrangian

$$\mathcal{L} = \sum_i \lambda_i - \frac{1}{2} \sum_{ij} \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{z}_j,$$

subject to the constraints $\lambda_i \geq 0$, $\sum_i \lambda_i y_i = 0$ in terms of a convex optimization problem

$$\frac{1}{2} \boldsymbol{\lambda}^T \begin{bmatrix} y_1 y_1 K(\mathbf{x}_1, \mathbf{x}_1) & y_1 y_2 K(\mathbf{x}_1, \mathbf{x}_2) & \dots & \dots & y_1 y_n K(\mathbf{x}_1, \mathbf{x}_n) \\ y_2 y_1 K(\mathbf{x}_2, \mathbf{x}_1) & y_2 y_2 K(\mathbf{x}_2, \mathbf{x}_2) & \dots & \dots & y_2 y_n K(\mathbf{x}_2, \mathbf{x}_n) \\ \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots \\ y_n y_1 K(\mathbf{x}_n, \mathbf{x}_1) & y_n y_2 K(\mathbf{x}_n, \mathbf{x}_2) & \dots & \dots & y_n y_n K(\mathbf{x}_n, \mathbf{x}_n) \end{bmatrix} \boldsymbol{\lambda} - \mathbb{1}^T \boldsymbol{\lambda},$$

subject to $\mathbf{y}^T \boldsymbol{\lambda} = 0$. Here we defined the vectors

$\boldsymbol{\lambda} = [\lambda_1, \lambda_2, \dots, \lambda_n]$ and $\mathbf{y} = [y_1, y_2, \dots, y_n]$. If we add the slack constants this leads to the additional constraint $0 \leq \lambda_i \leq C$.

Convex optimization

We can rewrite this (see the solutions below) in terms of a convex optimization problem of the type

$$\begin{aligned} \min_{\lambda} \quad & \frac{1}{2} \boldsymbol{\lambda}^T \mathbf{P} \boldsymbol{\lambda} + \mathbf{q}^T \boldsymbol{\lambda}, \\ \text{subject to} \quad & \mathbf{G} \boldsymbol{\lambda} \preceq \mathbf{h} \quad \wedge \quad \mathbf{A} \boldsymbol{\lambda} = \mathbf{f}. \end{aligned}$$

Below we discuss how to solve these equations. Here we note that the matrix $\mathbf{P}$ has matrix elements $p_{ij} = y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$. Given a kernel K and the targets y_i this matrix is easy to set up. The constraint $\mathbf{y}^T \boldsymbol{\lambda} = 0$ leads to $f = 0$ and $\mathbf{A} = \mathbf{y}$. How to set up the matrix $\mathbf{G}$ is discussed later. Here note that the inequalities $0 \leq \lambda_i \leq C$ can be split up into $0 \leq \lambda_i$ and $\lambda_i \leq C$. These two inequalities define then the matrix $\mathbf{G}$ and the vector $\mathbf{h}$.

Different kernels

There are several popular kernels being used. These are

1. Linear: $K(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T \mathbf{y}$,
2. Polynomial: $K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y} + \gamma)^d$,
3. Gaussian Radial Basis Function:
 $K(\mathbf{x}, \mathbf{y}) = \exp(-\gamma \|\mathbf{x} - \mathbf{y}\|^2)$,
4. Tanh: $K(\mathbf{x}, \mathbf{y}) = \tanh(\mathbf{x}^T \mathbf{y} + \gamma)$,

and many other ones.

Mercer's theorem

An important theorem for us is [Mercer's theorem](#). The theorem states that if a kernel function K is symmetric, continuous and leads to a positive semi-definite matrix $\mathbf{P}$ then there exists a function ϕ that maps $\mathbf{x}_i$ and $\mathbf{x}_j$ into another space (possibly with much higher dimensions) such that

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j).$$

So you can use K as a kernel since you know ϕ exists, even if you don't know what ϕ is.

Note that some frequently used kernels (such as the Sigmoid kernel) don't respect all of Mercer's conditions, yet they generally work well in practice.

The moons example

```
from __future__ import division, print_function, unicode_literals
```

```
import numpy as np
np.random.seed(42)
```

```
import matplotlib
import matplotlib.pyplot as plt
plt.rcParams['axes.labelsize'] = 14
plt.rcParams['xtick.labelsize'] = 12
plt.rcParams['ytick.labelsize'] = 12
```

```
from sklearn.svm import SVC
from sklearn import datasets
```

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC
```

```
from sklearn.datasets import make_moons
X, y = make_moons(n_samples=100, noise=0.15, random_state=42)
```

```
def plot_dataset(X, y, axes):
    plt.plot(X[:, 0][y==0], X[:, 1][y==0], "bs")
    plt.plot(X[:, 0][y==1], X[:, 1][y==1], "g^")
```

Mathematical optimization of convex functions

A mathematical (quadratic) optimization problem, or just optimization problem, has the form

$$\begin{aligned} \min_{\lambda} \quad & \frac{1}{2} \boldsymbol{\lambda}^T \mathbf{P} \boldsymbol{\lambda} + \mathbf{q}^T \boldsymbol{\lambda}, \\ \text{subject to} \quad & \mathbf{G} \boldsymbol{\lambda} \preceq \mathbf{h} \wedge \mathbf{A} \boldsymbol{\lambda} = \mathbf{f}. \end{aligned}$$

subject to some constraints for say a selected set $i = 1, 2, \dots, n$. In our case we are optimizing with respect to the Lagrangian multipliers λ_i , and the vector $\boldsymbol{\lambda} = [\lambda_1, \lambda_2, \dots, \lambda_n]$ is the optimization variable we are dealing with.

In our case we are particularly interested in a class of optimization problems called convex optimization problems.

Convex optimization problems play a central role in applied mathematics and we recommend strongly [Boyd and Vandenberghe's text on the topics](#).

How do we solve these problems?

If we use Python as programming language and wish to venture beyond **scikit-learn**, **tensorflow** and similar software which makes our lives so much easier, we need to dive into the wonderful world of quadratic programming. We can, if we wish, solve the minimization problem using say standard gradient methods or conjugate gradient methods. However, these methods tend to exhibit a rather slow converge. So, welcome to the promised land of quadratic programming.

The functions we need are contained in the [quadratic programming library CVXOPT](#) and we need to import it together with **numpy** as

```
import numpy
import cvxopt
```

This will make our life much easier. You don't need to write your own optimizer.

A simple example

We remind ourselves about the general problem we want to solve

$$\begin{aligned} \min_{\mathbf{x}} \quad & \frac{1}{2} \mathbf{x}^T \mathbf{P} \mathbf{x} + \mathbf{q}^T \mathbf{x}, \\ \text{subject to} \quad & \mathbf{G} \mathbf{x} \preceq \mathbf{h} \wedge \mathbf{A} \mathbf{x} = \mathbf{f}. \end{aligned}$$

Let us show how to perform the optimization using a simple case.
Assume we want to optimize the following problem

$$\begin{aligned} \min_{x,y} \quad & \frac{1}{2} x^2 + 5x + 3y \\ \text{subject to} \quad & \\ & x, y \geq 0 \\ & x + 3y \geq 15 \\ & 2x + 5y \leq 100 \\ & 3x + 4y \leq 80. \end{aligned}$$

Rewriting in terms of vectors and matrices

The minimization problem can be rewritten in terms of vectors and matrices as (with x and y being the unknowns)

$$\frac{1}{2} \begin{bmatrix} x \\ y \end{bmatrix}^T \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 3 \\ 4 \end{bmatrix}^T \begin{bmatrix} x \\ y \end{bmatrix}.$$

Similarly, we can now set up the inequalities (we need to change $\geq$ to $\leq$ by multiplying with -1 on bot sides) as the following matrix-vector equation

$$\begin{bmatrix} -1 & 0 \\ 0 & -1 \\ -1 & -3 \\ 2 & 5 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \preceq \begin{bmatrix} 0 \\ 0 \\ -15 \\ 100 \\ 80 \end{bmatrix}.$$

Rewriting inequalities

We have collapsed all the inequalities into a single matrix $\mathbf{G}$. We see also that our matrix

$$\mathbf{P} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

is clearly positive semi-definite (all eigenvalues larger or equal zero). Finally, the vector $\mathbf{h}$ is defined as

$$\mathbf{h} = \begin{bmatrix} 0 \\ 0 \\ -15 \\ 100 \\ 80 \end{bmatrix}.$$

Since we don't have any equalities the matrix $\mathbf{A}$ is set to zero The following code solves the equations for us

```
# Import the necessary packages
import numpy
from cvxopt import matrix
from cvxopt import solvers
P = matrix(numpy.diag([1,0]), tc='d')
q = matrix(numpy.array([3,4]), tc='d')
G = matrix(numpy.array([[ -1,0],[0,-1],[ -1,-3],[2,5],[3,4]]), tc='d')
```

Back to the more realistic cases

We are now ready to return to our setup of the optimization problem for a more realistic case. Introducing the **slack** parameter C we have

$$\frac{1}{2} \boldsymbol{\lambda}^T \begin{bmatrix} y_1 y_1 K(\mathbf{x}_1, \mathbf{x}_1) & y_1 y_2 K(\mathbf{x}_1, \mathbf{x}_2) & \dots & \dots & y_1 y_n K(\mathbf{x}_1, \mathbf{x}_n) \\ y_2 y_1 K(\mathbf{x}_2, \mathbf{x}_1) & y_2 y_2 K(\mathbf{x}_2, \mathbf{x}_2) & \dots & \dots & y_2 y_n K(\mathbf{x}_2, \mathbf{x}_n) \\ \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots \\ y_n y_1 K(\mathbf{x}_n, \mathbf{x}_1) & y_n y_2 K(\mathbf{x}_n, \mathbf{x}_2) & \dots & \dots & y_n y_n K(\mathbf{x}_n, \mathbf{x}_n) \end{bmatrix} \boldsymbol{\lambda} - \mathbb{I} \boldsymbol{\lambda},$$

subject to $\mathbf{y}^T \boldsymbol{\lambda} = 0$. Here we defined the vectors

$\boldsymbol{\lambda} = [\lambda_1, \lambda_2, \dots, \lambda_n]$ and $\mathbf{y} = [y_1, y_2, \dots, y_n]$. With the slack constants this leads to the additional constraint $0 \leq \lambda_i \leq C$.

Using the **CVXOPT** library, the matrix P would then be defined by the above matrix while the KKT conditions would all be collected by the matrix G .

Quantum SVMs

The idea of a classical SVM is that we have a set of points that are in either one group or another and we want to find a line that separates these two groups. This line can be linear, but it can also be much more complex, which can be achieved by the use of Kernels.

What will we need in the case of a quantum computer?

We will have to translate the classical data point $\vec{x}$ into a quantum datapoint $|\Phi(\vec{x})\rangle$. This can be achieved by a circuit $\mathcal{U}_{\Phi(\vec{x})}|0\rangle$. Here $\Phi()$ could be any classical function applied on the classical data $\vec{x}$.

We need a parameterized quantum circuit $W(\theta)$ that processes the data in a way that in the end we can apply a measurement that returns a classical value -1 or 1 for each classical input $\vec{x}$ that identifies the label of the classical data.

The most general ansatz

Following these steps we can define an ansatz for this kind of problem which is

$$W(\theta)\mathcal{U}_{\Phi}(\vec{x})|0\rangle.$$

These kind of ansatzes are called quantum variational circuits.

Quantum SVM

In the case of a quantum SVM we will only use the quantum feature maps

$$\mathcal{U}_{\Phi(\vec{x})},$$

to translate the classical data into quantum states and build the Kernel of the SVM out of these quantum states. After calculating the Kernel matrix on the quantum computer we can train the Quantum SVM the same way as the classical SVM.

Defining the Quantum Kernel

The idea of the quantum kernel is exactly the same as in the classical case. We take the inner product

$$K(\vec{x}, \vec{z}) = |\langle \Phi(\vec{x}) | \Phi(\vec{z}) \rangle|^2 = \langle 0^n | \mathcal{U}_{\Phi(\vec{x})}^\dagger \mathcal{U}_{\Phi(\vec{z})} | 0^n \rangle,$$

but now with the quantum feature maps

$$\mathcal{U}_{\Phi(\vec{x})}.$$

The idea is that if we choose a quantum feature maps that is not easy to simulate with a classical computer we might obtain a quantum advantage.

Side note

There is no proof yet that the QSVM brings a quantum advantage, but the argument the authors of <https://arxiv.org/pdf/1804.11326.pdf> make, is that there is for sure no advantage if we use feature maps that are easy to simulate classically, because then we would not need a quantum computer to construct the Kernel.

Feature map

For the feature maps we use the ansatz

$$\mathcal{U}_{\Phi(x)} = U_{\Phi(x)} \otimes H^{\otimes n},$$

where

$$U_{\Phi(x)} = \exp \left(i \sum_{S \in n} \phi_S(x) \prod_{i \in S} Z_i \right),$$

which simplifies a lot when we (like in

<https://arxiv.org/pdf/1804.11326.pdf>) only consider $S \leq 2$ interactions, which means we only let two qubits interact at a time. For $S \leq 2$ the product $\prod_{i \in S}$ only leads to interactions $Z_i Z_j$ and non interacting terms Z_i . And the sum $\sum_{S \in n}$ over all these terms that are possible with n qubits.

Classical functions

Finally we define the classical functions $\phi_i(\vec{x}) = x_i$ and

$$\phi_{i,j}(\vec{x}) = (\pi - x_i)(\pi - x_j).$$

If we write this ansatz for 2 qubits and $S \leq 2$ we see how it simplifies:

$$U_{\Phi(x)} = \exp(i(x_1 Z_1 + x_2 Z_2 + (\pi - x_1)(\pi - x_2) Z_1 Z_2)).$$

We won't get into details too much here, why we would take this ansatz. It is simply an ansatz that is simple enough and leads to good results.

Finally we can define a depth of these circuits. Depth 2 means we repeat this ansatz two times. Which means our feature map becomes $U_{\Phi(x)} \otimes H^{\otimes n} \otimes U_{\Phi(x)} \otimes H^{\otimes n}$

Plans for next week

1. Discussion of quantum support vector machines and quantum kernels
2. Introducing quantum neural networks